

Intro to DL4MSc: NLP - Tokenization, Embeddings

Alexander Binder

October 24, 2025

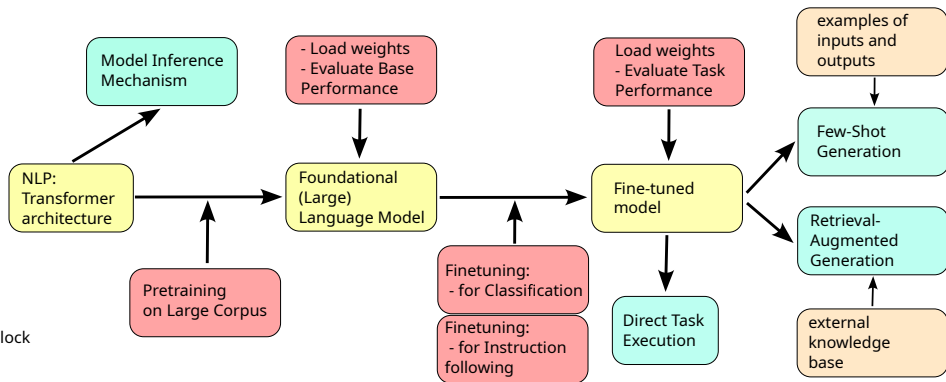
- 1 The high level plan
- 2 Tokenization
- 3 BPE
- 4 Dataset class example for next token prediction
- 5 Token embeddings (simplified)
- 6 Position encodings
- 7 Position encodings

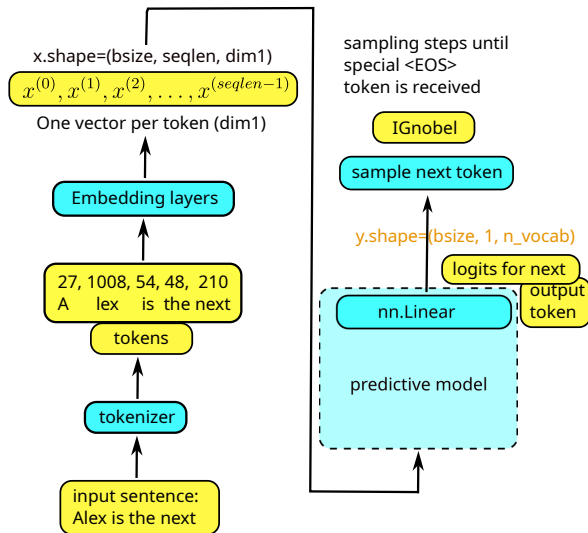
Data Preparation:

- Tokenization
 - Encoding
 - Decoding
- Dataset class
 - Minibatch Sampling

Arch:

- Embedding Layer
- Attention Layer
- Decoder Transformer Block





- ① The high level plan
- ② Tokenization
- ③ BPE
- ④ Dataset class example for next token prediction
- ⑤ Token embeddings (simplified)
- ⑥ Position encodings
- ⑦ Position encodings

Preprocessing goals in NLP

- Input: a text, a sequence of words and punctuation marks
- Goal: map it into a sequence of vectors

Tasks:

- split text into words and punctuation marks
 - add special tokens like <UNK>, <EOS>, 4x space
- map words / punctuation marks onto different indices or one-hot vectors
- improved handling of grammar-induced variations of words: Byte-Pair encoding

Task:

- split text into words and punctuation marks
 - python re package, re.split(...)

```
txt = 'This is: a biz blabla. More blubb here.'  
res=re.split(r"token1|token2|token3",txt)  
res2=re.split(r"(token1|token2|token3)",txt)  
res=re.split(r"\s|[\.]|AND",txt)  
#. alone causes an error because it has a special meaning in re  
res2=re.split(r"(\s|[\.]|AND)",txt)
```

Task:

- ⦿ split text into words and punctuation marks

```
txt = 'This is: a biz blabla. More blubb here.'
```

```
res2=re.split(r"([.:]|\s)",txt) #[] encloses multiple single-char patterns  
res2 = [ item.strip() for item in res2 if item.strip()]
```


Task:

- split text into words and punctuation marks

```
txt = 'This is: a biz blabla. More blubb here.'
```

```
res2=re.split(r'([,.;?_!"()\\[\]\']|--|\s)',txt) #note the \c to escape special chars  
res2 = [ item.strip() for item in res2 if item.strip()]
```

- a subtle matter: the [in the regex does not have to be escaped, the] has to be, in order to avoid telling it that we have closed the [at the start.

Task:

- in order to map words to indices, build a vocabulary of all encountered words in the training corpus

```
listoftxt =[ 'This is: a biz blabla. ', 'I am a cat.', 'Dogs are dumb.' ]
```

```
allwords = set()
```

```
for txt in listoftxt:
```

```
    res2=re.split(r'([.,:;?_!"()[\]\']|--|\s)',txt) #note the \c to escape special chars c
```

```
    res2 = [ item.strip() for item in res2 if item.strip()]
```

```
    allwords.update(res2) #for adding multiple elements to a set
```

```
#maybe one wants to sort the vocabulary
```

```
#vocab = {}
```

```
#counter =-1
```

```
#for word in allwords:
```

```
#   counter+=1
```

```
#   vocab[word]=counter
```

```
vocab = { word:ind for ind,word in enumerate(allwords) }
```

```
print(vocab)
```

Some texts to try:

[https://en.wikisource.org/wiki/The_Works_of_Lord_Byron_\(ed._Coleridge,_Prothero\)/Poetry/Volume_1/The_First_Kiss_of_Love](https://en.wikisource.org/wiki/The_Works_of_Lord_Byron_(ed._Coleridge,_Prothero)/Poetry/Volume_1/The_First_Kiss_of_Love)

To see what can go wrong with the current version:

[https://en.wikisource.org/wiki/Consolidated_Fund_\(No._2\)_Act_2004](https://en.wikisource.org/wiki/Consolidated_Fund_(No._2)_Act_2004)

something more dry:

https://en.wikisource.org/wiki/The_Harveian_Oration_1893

Copy-paste it manually or set a user-agent for downloaders

https://foundation.wikimedia.org/wiki/Policy:Wikimedia_Foundation_User-Agent_Policy

- ⦿ What to do if at inference (or training time) time one encounters a word out of the vocabulary ?

Expect it to happen:

https://en.wikipedia.org/wiki/List_of_Unicode_characters

https://en.wikipedia.org/wiki/Emoji#Unicode_blocks

- ⦿ add a $\langle \text{UNK} \rangle$ token to map such words onto it
- ⦿ some other tokens might be useful like $\langle \text{ENDOFTEXT} \rangle$ to tell the model that this was all and it can start to produce an output

Task:

- in order to map words to indices, build a vocabulary of all encountered words in the training corpus

```
listoftxt =[ 'This is: a biz blabla. ', 'I am a cat.', 'Dogs are dumb.' ]
```

```
allwords = set()
```

```
for txt in listoftxt:
```

```
    res2=re.split(r'([,.;?_!"() [\]\']|--|\s)',txt) #note the \c to escape special chars c
```

```
    res2 = [ item.strip() for item in res2 if item.strip()]
```

```
    allwords.update(res2) #for adding multiple elements to a set
```

```
allwords.update([ '<|UNK|>' , '<|ENDOFTEXT|>', '<|THISSUCKS|>' ])
```

```
#faster:
```

```
vocab = { word:ind for ind,word in enumerate(allwords) }
```

```
print(vocab)
```

- ⊙ have done as one-time-step: build a vocabulary of all encountered words in the training corpus
- ⊙ next step: code a tokenizer that uses the vocabulary

```
class tokenizer_simple_v1:
    def __init__(self, vocab):
        self.str2int = vocab
        self.int2str = { i:s for s,i in vocab.items()}

    def encode(self, text):
        preprocessed = re.split(r'([.,:;?_"()\\[\]\']|--|\s)', text)
        preprocessed = [ item.strip() for item in preprocessed if item.strip() ]
        #map unknowns to UNK
        #preprocessed2 = [item if item in self.str2int else '<|UNK|>' for item in preprocessed ]
        preprocessed2 = []
        for item in preprocessed:
            if item in self.str2int:
                preprocessed2.append(item)
            else:
                preprocessed2.append('<|UNK|>')

        ids = [self.str2int[s] for s in preprocessed2]
        return ids

    def decode(self, ids):
        text = " ".join([self.int2str[i] for i in ids])
        text = re.sub(r'\s+([.,:;?_"()\\[\]\']|--|\s)', r'\1', text) #removes spaces before this punctuation
        return text
```

- ⦿ have done as one-time-step: build a vocabulary of all encountered words in the training corpus
- ⦿ next step: code a tokenizer that uses the vocabulary

```
listoftxt1 = [ 'mysterious', 'cat, dog, Dogs, dumb,' , 'I am. You are', 'whatever a word', 'A'  
listoftxt2 =[ 'I am a cat. Dogs are dumb. A new word', ]
```

```
vocab = genvocab(listoftxt1)
```

```
tokenizer = tokenizer_simple_v1(vocab)
```

```
inputtext = listoftxt2[0]  
ids= tokenizer.encode(inputtext)
```

```
print(ids)  
outtext = tokenizer.decode(ids)  
print(inputtext)  
print(outtext)
```

exemplary ones:

- **common:** '`< |ENDOFTEXT| >`' '`< |EOS| >`'. Model is trained to predict it as a marker indicating that it is done generating. Use: One can stop the model inference, when one receives this token as output from the model.
- **common:** '`< |BOS| >`' beginning of sentence. Token inputted into a decoder model as the first input. It tells the model to start generating a response.
- special tokens indicating non-textual content in need of a separate parser like '`< |IMAGE_START| >`', '`< |IMAGE_END| >`', '`< |PDF_START| >`', '`< |PDF_END| >`'

others added often during instruction-fine tuning, for example

- one has collected data where a user is writing something, and a model is providing the desired response. One needs to mark what part is said by the user, and what by the model
 - '`< |user| >`', '`< |assistant| >`'
 - '`< |startofturn| >`', '`< |endofturn| >`' to mark the start and end of a response by user or assistant (model).

exemplary ones for instruction tuning

'< |startofturn| >' '< |user| >' What can I buy my aunt for a birthday gift. She loves to do Karate and goes to the Dojo 5 times each week. '< |endofturn| >'

'< |startofturn| >' '< |assistant| >' If your aunt is ok to receive a gift which is not a surprise you can ask her whether she needs a new hilt for her sword. '< |endofturn| >'

- ① The high level plan
- ② Tokenization
- ③ BPE**
- ④ Dataset class example for next token prediction
- ⑤ Token embeddings (simplified)
- ⑥ Position encodings
- ⑦ Position encodings

- deal with grammar variations, reduce the frequency of resorting to UNK tokens.
- learn a vocabulary

one example here: <https://huggingface.co/learn/llm-course/en/chapter6/5>

Example set of words and frequencies: ("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5)
... from <https://huggingface.co/learn/llm-course/en/chapter6/5>

- ⦿ init the vocabulary with all single letters present.
["b", "g", "h", "n", "p", "s", "u"]
- ⦿ init a data structure which contains:
 - the training set of words decomposed into the single letters, the frequency of each word
("h" "u" "g", 10), ("p" "u" "g", 5), ("p" "u" "n", 12), ("b" "u" "n", 4), ("h" "u" "g" "s", 5)

- find most frequent pair of adjacent elements in the training set of words:
("h" "u" "g", 10), ("p" "u" "g", 5), ("p" "u" "n", 12), ("b" "u" "n", 4), ("h" "u" "g" "s", 5)
it is "u" "g" with 20
- apply merging of elements in the vocabulary with the highest frequency in the training set of words
 - add merged result into the vocabulary
 - apply the merging to the words in the training set

Vocabulary: ["b", "g", "h", "n", "p", "s", "u", "ug"]

Corpus: ("h" "ug", 10), ("p" "ug", 5), ("p" "u" "n", 12), ("b" "u" "n", 4), ("h" "ug" "s", 5)

- iterate this until a max vocabulary size is reached (next is)

Python lib for BPE: tiktoken <https://github.com/openai/tiktoken>

```
enc = tiktoken.encoding_for_model("gpt-4o")
enc = tiktoken.encoding_for_model("gpt2")

ids= enc.encode("Only black roses for the vampire lady.")
print(ids)
out= enc.decode(ids)
```

other tokenizers exist like WordPiece <https://arxiv.org/abs/1609.08144> (different merging rules) or SentencePiece <https://arxiv.org/abs/1808.06226> (do not make assumption about space as separator).

- ① The high level plan
- ② Tokenization
- ③ BPE
- ④ Dataset class example for next token prediction
- ⑤ Token embeddings (simplified)
- ⑥ Position encodings
- ⑦ Position encodings

next: how to build a simplified dataset class for one particular prediction setup ?

Consider a setup of sequential next token prediction:

- ⦿ input a text: "After taking some Magnesium, doing exercises"
 - predict the next word of it: "makes" (wrong prediction)
- ⦿ input the text with the next ground truth word: "After taking some Magnesium, doing exercises feels"
 - predict again the next word of it: "easier"
 - predict again the next word of it: "indeed"
 - predict again the next word of it: "."
- ⦿ stop sequential prediction when one predicts an <'ENDOFTEXT'> token.

Consider a setup of sequential next token prediction:

- iterate this for another example:
 - The → vanilla
 - The vanilla → cake
 - The vanilla cake → with
 - The vanilla cake with → cherries
 - The vanilla cake with cherries → tasted
 - The vanilla cake with cherries tasted → like
 - The vanilla cake with cherries tasted like → heaven
 - The vanilla cake with cherries tasted like heaven → .
 - The vanilla cake with cherries tasted like heaven. → <'ENDOFTEXT'>

done by so-called autoregressive models. ChatGPT as an example. We will talk of a different setup in a later lecture.

Consider a setup of sequential next token prediction:

- ⦿ input a text: "After taking some Magnesium, doing exercises"
 - predict the next word of it: "makes" (wrong prediction)
 - ⦿ what parameters we may use given a single sequence of ids to create a data set for such a task ?
 - maxlength: how long sequences to consider
 - stride: how many tokens to move to create the next pair of (x, y) of input and predicted label
- in the above we moved always one word, stride= 1

as a single function:

```
def txt2inputsandlabels( txt, tokenizer, maxlen, stride=1):
    input_ids = []
    target_ids = []

    token_ids = tokenizer.encode(txt)

    #the important loop over the text
    for i in range(0, len(token_ids) - maxlen, stride):
        input_chunk = token_ids[i:i + maxlen]
        target_chunk = token_ids[i + 1: i + maxlen + 1] #labels are shifted by 1
        input_ids.append(torch.tensor(input_chunk))
        target_ids.append(torch.tensor(target_chunk))

    return input_ids, target_ids
```

next: make it part of an init of a `torch.utils.data.Dataset` class

```
class nexttokenpred_simplesdataset(torch.utils.data.Dataset):

    def __init__(self, listoftxt, tokenizer, maxlen, stride=1):
        self.input_ids = []
        self.target_ids = []

        for txt in listoftxt:
            token_ids = tokenizer.encode(txt)
            #the important loop over the text
            for i in range(0, len(token_ids) - maxlen, stride):
                input_chunk = token_ids[i:i + maxlen]
                target_chunk = token_ids[i + 1: i + maxlen + 1] #labels are shifted by 1
                self.input_ids.append(torch.tensor(input_chunk))
                self.target_ids.append(torch.tensor(target_chunk))

    def __len__(self):
        return len(self.input_ids)

    def __getitem__(self, idx):
        return self.input_ids[idx], self.target_ids[idx]
```

next step: wrap it into a dataloader to be able to create minibatches

- ① The high level plan
- ② Tokenization
- ③ BPE
- ④ Dataset class example for next token prediction
- ⑤ Token embeddings (simplified)
- ⑥ Position encodings
- ⑦ Position encodings

goal: map an id to a vector

- have an so called embedding layer, which has a trainable weight matrix U of shape $U.shape = (vocab_size, embed_dim)$
- the mapping: $id \mapsto U[id, :]$
- `torch.nn.Embedding`
- learn W during training to get similarities between semantically close words

To visualize the weight matrix after random initialisation:

```
torch.manual_seed(3)

vocabsize = 6
embeddim = 3
embedding_layer = torch.nn.Embedding(vocabsize, embeddim)

print(embedding_layer.weight)
```

- ① The high level plan
- ② Tokenization
- ③ BPE
- ④ Dataset class example for next token prediction
- ⑤ Token embeddings (simplified)
- ⑥ Position encodings
- ⑦ Position encodings

Problem:

- order of words matters in a sentence
- Embedding as above does not contain an information about where a word (or token) appears in a sentence
- idea: add a vector to the word embedding which contains such an information
 - simplest special case: absolute positional encodings $PE(i)$ – a function of the position pos of the token in the sequence which returns a vector which is equal to the dimension of the word embeddings. Let id be a token with position pos in the sequence of tokens. Let U be the embedding for all tokens. Then:

$$id \mapsto U[id, :]$$
$$f(id) = U[id, :] + PE(pos)$$

$f(id)$ is then the feature used in the neural network

- ① The high level plan
- ② Tokenization
- ③ BPE
- ④ Dataset class example for next token prediction
- ⑤ Token embeddings (simplified)
- ⑥ Position encodings
- ⑦ Position encodings

The original NLP transformer paper (Vaswani et al. Attention is all you need. NIPS 2017) https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf used predefined position encodings. Let d be the dimensionality of the word embeddings and i the position i in the sequence of tokens

$$PE(i, k) = \begin{cases} \sin\left(\frac{i}{(10^4)^{k/d}}\right) & \text{if } 0 \leq k \leq d \text{ is even} \\ \cos\left(\frac{i}{(10^4)^{(k-1)/d}}\right) & \text{if } 0 \leq k \leq d \text{ is odd} \end{cases}$$

$$PE(i, 2l) = \sin\left(\frac{i}{10^{8l/d}}\right)$$

$$PE(i, 2l + 1) = \cos\left(\frac{i}{10^{8l/d}}\right)$$

- alternative 1: trainable absolute position encodings. Let d be the dimensionality of the word embeddings and i the position i in the sequence of tokens

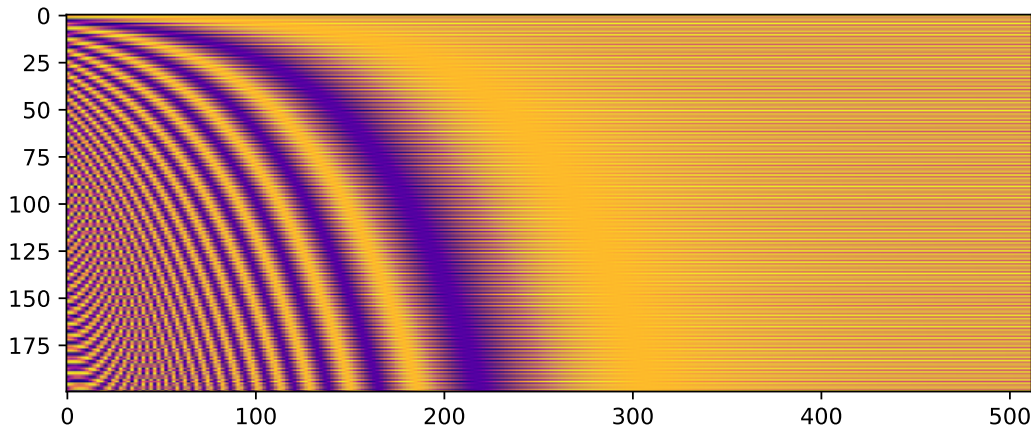
$$PE(i, k) = V[i, k], \quad 0 \leq k \leq d$$

$$PE(i, :) = V[i, :], \text{ as vector}$$

V is a trainable matrix of shape $V.shape = (num_positions, embed_dim)$

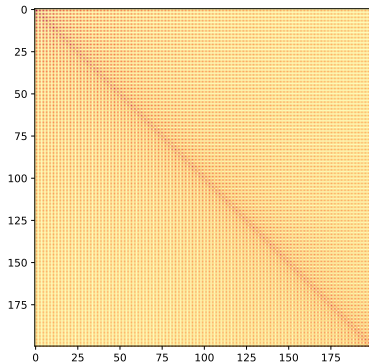
- implementation again using `torch.nn.Embedding`, but the input is not the token id. It is the token position.

How do they look like ?



How do they look like ?

plotting the Vaswani et al fixed position encodings is less informative than plotting the pairwise similarities between two encodings at different positions.



How do they look like ?

This plots pair-wise similarities between position encodings at two different positions:

<https://arxiv.org/pdf/2010.04903>

More types of position encodings will be explained, when we have introduced the attention layer.